

Intro Hugging Face and Python

Dirk Wulff & Zak Hussain



MAX PLANCK INSTITUTE
FOR HUMAN DEVELOPMENT



OUR SOFTWARE STACK



+



+



OUR SOFTWARE STACK



+



+



WHY IS HUGGING FACE?



WHY IS HUGGING FACE?



Traditional language modelling pipeline:

WHY IS HUGGING FACE?



Traditional language modelling pipeline:

1. Find out the model architecture

WHY IS HUGGING FACE?



Traditional language modelling pipeline:

1. Find out the model architecture
2. Implement the model architecture in code with deep learning frameworks (e.g PyTorch/Tensorflow).

1. DEEP LEARNING LIBRARIES CAN BE DIFFICULT

1. DEEP LEARNING LIBRARIES CAN BE DIFFICULT

I  KING HATE TENSORFLOW #53549

✓ Closed

ghost opened this issue 9 hours ago · 2 comments

WHY HUGGING FACE?

Traditional language modelling pipeline:

1. Find out the model architecture
2. Implement the model architecture in code with deep learning libraries (e.g PyTorch/Tensorflow).

WHY HUGGING FACE?

Traditional language modelling pipeline:

1. Find out the model architecture
2. Implement the model architecture in code with deep learning libraries (e.g PyTorch/Tensorflow).
3. Load the pretrained weights (if available) from a server.
4. Process the inputs (using the correct tokenizer for the model)
5. Implement data loaders
6. Define a loss function
7. Stick a task-specific “head” on the model

HUGGING FACE PIPELINES

Import pipeline

```
from transformers import pipeline
```

Initialise
pipeline

```
pipe = pipeline('text-generation', model='gpt2')
```

Load model
input

```
prompt = """
```

```
Once upon a time in a land far far away, there was a young prince named  
John. He was known for his bravery and courage. One day, he decided to go on  
an adventure to explore the unknown lands.
```

```
"""
```

Feed input
the model

```
output = pipe(prompt, max_length=100)
```

```
print(output)
```

HUGGING FACE

 **Hugging Face**

Models Datasets Spaces Buckets **NEW** Docs Pricing 

Main Tasks Libraries Languages Licenses Other

Tasks

Text Generation Any-to-AnyImage-Text-to-TextImage-to-ImageText-to-ImageText-to-VideoText-to-Speech+ 44

Parameters

< 1B 6B 12B 32B 128B > 500B

Libraries

PyTorch TensorFlow JAXTransformers Diffusers GGUFMLX Transformers.js Safetensors sentence-transformers ONNX + 42

Apps

vLLM llama.cpp MLX LLM LM Studio Ollama Jan Draw Things + 9

Inference Providers

Models 2,976,921 ☐ Base only ☒ Inference Available

yuxinlu1/gemma-4-12B-coder-fable5-composer2.5-v1-GGUF

Text Generation · 12B · Updated about 13 hours ago · 147k · 1.32k

google/diffusiongemma-26B-A4B-it

Image-Text-to-Text · 26B · Updated 7 days ago · 460k · 962

MiniMaxAI/MiniMax-M3

Image-Text-to-Text · 427B · Updated 1 day ago · 42.2k · 1.05k

moonshotai/Kimi-K2.7-Code

Image-Text-to-Text · 1.1T · Updated 2 days ago · 173k · 826

zai-org/GLM-5.2

Text Generation · 753B · Updated about 2 hours ago · 666 · 795

nvidia/LocateAnything-3B

Image-Text-to-Text · 4B · Updated 5 days ago · 130k · 2.12k

prefeitura-rio/Rio-3.5-Open-397B

Image-Text-to-Text · 403B · Updated 3 days ago · 190k · 317

unsloth/diffusiongemma-26B-A4B-it-GGUF

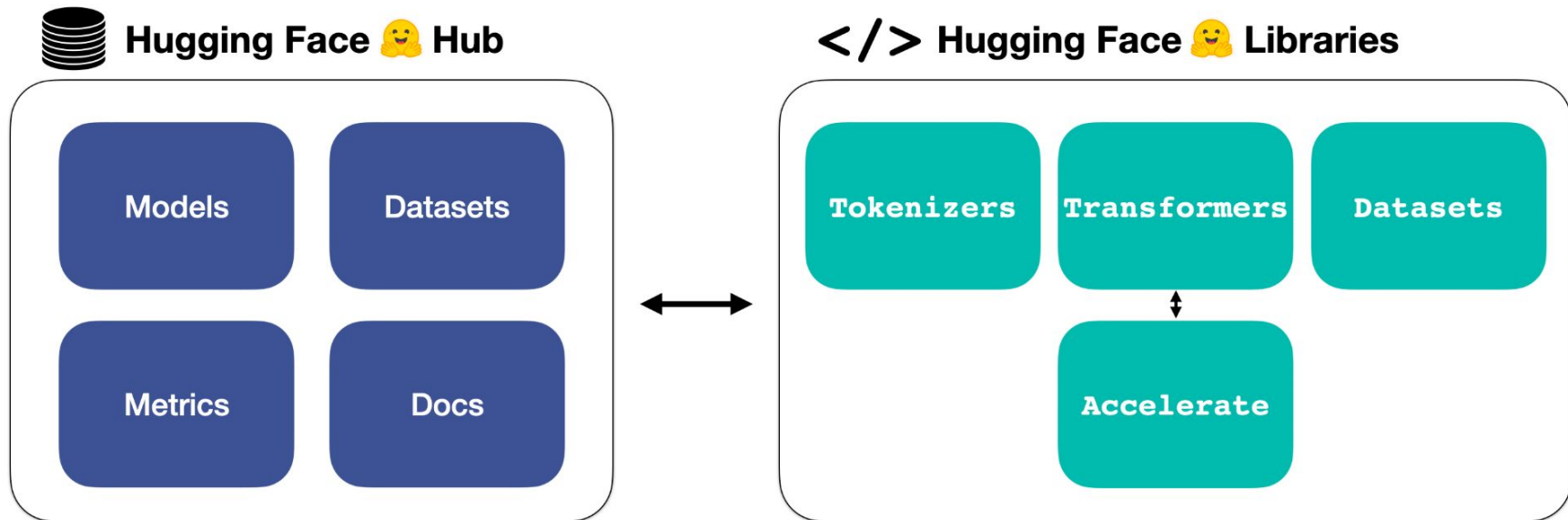
Image-Text-to-Text · 25B · Updated 5 days ago · 137k · 293

WeiboAI/VibeThinker-3B

Text Generation · 3B · Updated about 8 hours ago · 1.95k · 243

Wulff & Hussain | GSERM St Gallen 2026

THE HUGGING FACE ECOSYSTEM



HUGGING FACE DOCUMENTATION

Documentations

🔍 Search across all docs

• Hub

Host Git-based models, datasets and Spaces on the Hugging Face Hub.

• Hub Python Library

Client library for the HF Hub: manage repositories from your Python runtime.

• Inference API

Use more than 50k models through our public inference API, with scalability built-in.

• Transformers

State-of-the-art ML for Pytorch, TensorFlow, and JAX.

• Datasets

Access and share datasets for computer vision, audio, and NLP tasks.

• Huggingface.js

A collection of JS libraries to interact with Hugging Face, with TS types included.

• Inference Endpoints

Easily deploy your model to production on dedicated, fully managed infrastructure.

• Diffusers

State-of-the-art diffusion models for image and audio generation in PyTorch.

• Gradio

Build machine learning demos and other web apps, in just a few lines of Python.

• Transformers.js

Community library to run pretrained models from Transformers in your browser.

• PEFT

Parameter efficient finetuning methods for large models

OUR SOFTWARE STACK



+



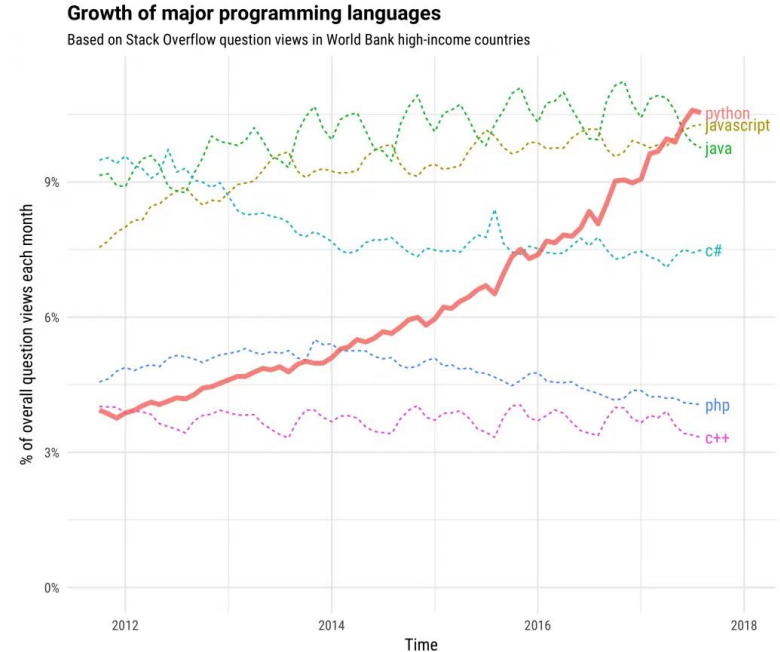
+







- A **high-level** programming language known for its simplicity and readability.
- Used in various domains such as web development, data analysis, **artificial intelligence**, and scientific computing.



OUR SOFTWARE STACK



+



+



PYTHON  + Google Colab 

PYTHON + Google Colab

 day_1.ipynb ☆

File Edit View Insert Runtime Tools Help [All changes saved](#)

+ Code + Text

RAM
Disk

 [6]

```
import pandas as pd
from sentence_transformers import SentenceTransformer
from transformers import pipeline
```

 Feature Extraction

We will begin by extracting features (numerical representations) from the text data using the `sentence-transformers` package. We will use the following three sentences, stored as a list of strings, as input to the model:

 [7]

```
sentences = [
    "I feel great this morning",
    "I am feeling very good today",
    "I am feeling terrible"
]
```

We will use the `all-MiniLM-L6-v2` model to extract features from the sentences. The model will encode the sentences into a 384-dimensional vector representation. We will then print the features as a pandas dataframe for easy viewing.



```
# Load the pre-trained model
model = SentenceTransformer('all-MiniLM-L6-v2')

# Extract features
features = model.encode(sentences)

# Print the features as a pandas dataframe
pd.DataFrame(features, index=sentences)
```

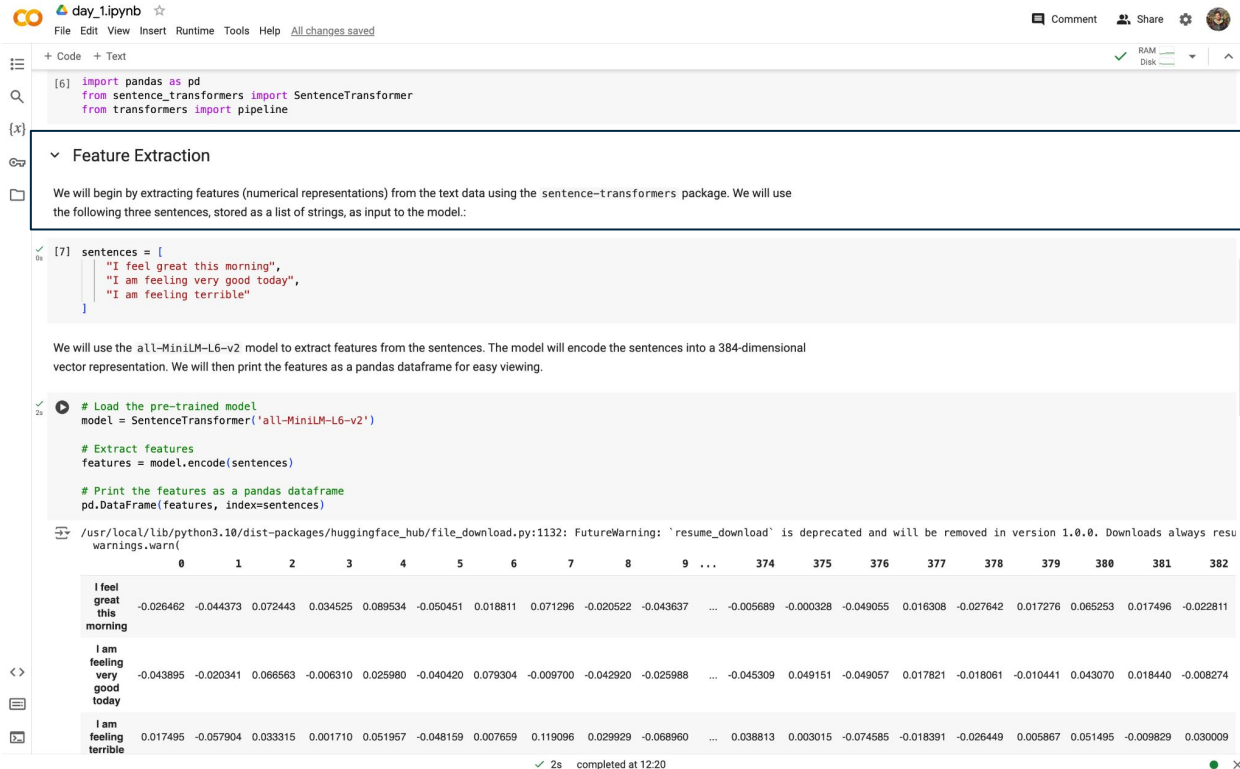
`/usr/local/lib/python3.10/dist-packages/huggingface_hub/file_download.py:1132: FutureWarning: 'resume_download' is deprecated and will be removed in version 1.0.0. Downloads always resu`

	0	1	2	3	4	5	6	7	8	9	...	374	375	376	377	378	379	380	381	382
I feel great this morning	-0.026462	-0.044373	0.072443	0.034525	0.089534	-0.050451	0.018811	0.071296	-0.020522	-0.043637	...	-0.005689	-0.000328	-0.049055	0.016308	-0.027642	0.017276	0.065253	0.017496	-0.022811
I am feeling very good today	-0.043895	-0.020341	0.068563	-0.006310	0.025980	-0.040420	0.079304	-0.009700	-0.042920	-0.025988	...	-0.045309	0.049151	-0.049057	0.017821	-0.018061	-0.010441	0.043070	0.018440	-0.008274
I am feeling terrible	0.017495	-0.057904	0.033315	0.001710	0.051957	-0.048159	0.007659	0.119096	0.029929	-0.068960	...	0.038813	0.003015	-0.074585	-0.018391	-0.026449	0.005867	0.051495	-0.009829	0.030009

2s completed at 12:20

PYTHON + Google Colab

1. Markdown



The screenshot shows a Google Colab notebook titled "day_1.ipynb". The interface includes a top bar with "File", "Edit", "View", "Insert", "Runtime", "Tools", and "Help" menus, along with a "Comment" button and a "Share" button. The notebook has two cells: a markdown cell and a code cell.

Markdown Cell:

Feature Extraction

We will begin by extracting features (numerical representations) from the text data using the `sentence-transformers` package. We will use the following three sentences, stored as a list of strings, as input to the model:

Code Cell:

```
[6] import pandas as pd
from sentence_transformers import SentenceTransformer
from transformers import pipeline
```

```
[7] sentences = [
    "I feel great this morning",
    "I am feeling very good today",
    "I am feeling terrible"
]
```

We will use the `all-MiniLM-L6-v2` model to extract features from the sentences. The model will encode the sentences into a 384-dimensional vector representation. We will then print the features as a pandas dataframe for easy viewing.

```
# Load the pre-trained model
model = SentenceTransformer('all-MiniLM-L6-v2')

# Extract features
features = model.encode(sentences)

# Print the features as a pandas dataframe
pd.DataFrame(features, index=sentences)
```

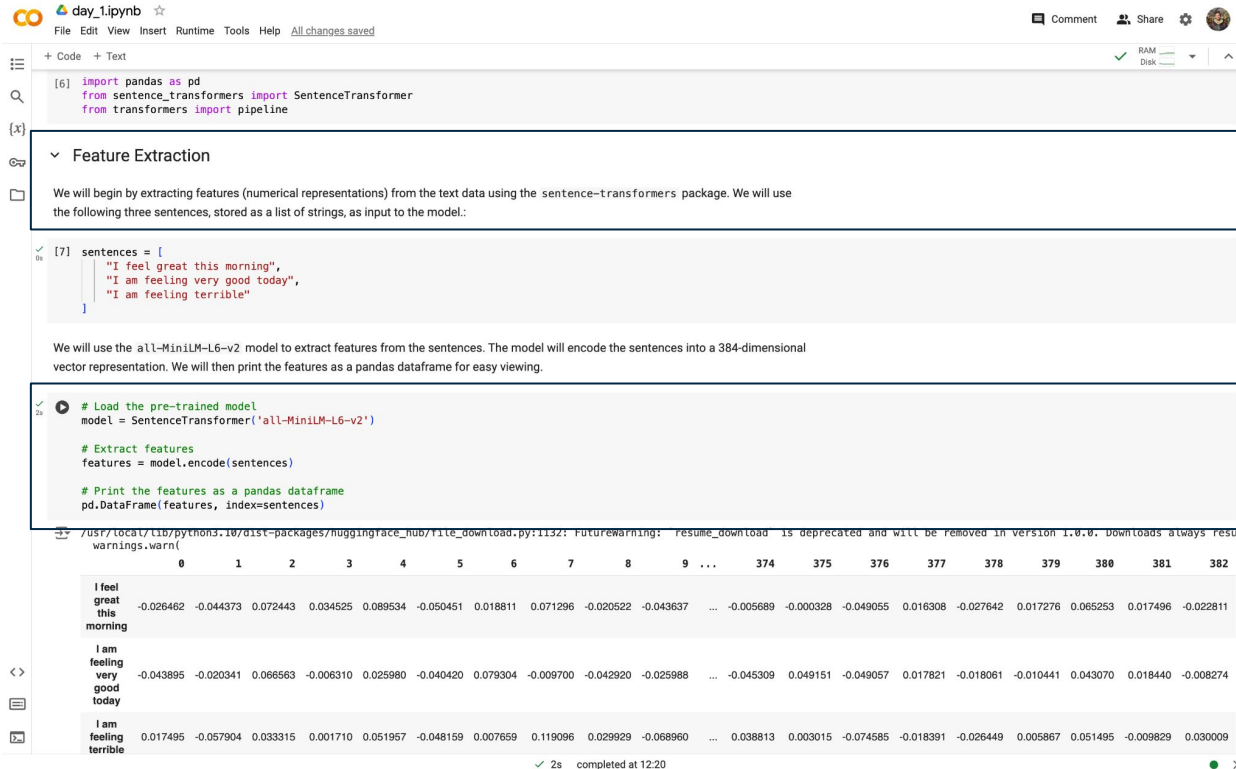
Warning: `resume_download` is deprecated and will be removed in version 1.0.0. Downloads always resume when possible. See: https://github.com/huggingface/huggingface_hub/pull/1317

	0	1	2	3	4	5	6	7	8	9	...	374	375	376	377	378	379	380	381	382
I feel great this morning	-0.026462	-0.044373	0.072443	0.034525	0.089534	-0.050451	0.018811	0.071296	-0.020522	-0.043637	...	-0.005689	-0.000328	-0.049055	0.016308	-0.027642	0.017276	0.065253	0.017496	-0.022811
I am feeling very good today	-0.043895	-0.020341	0.066563	-0.006310	0.025980	-0.040420	0.079304	-0.009700	-0.042920	-0.025988	...	-0.045309	0.049151	-0.049057	0.017821	-0.018061	-0.010441	0.043070	0.018440	-0.008274
I am feeling terrible	0.017495	-0.057904	0.033315	0.001710	0.051957	-0.048159	0.007659	0.119096	0.029929	-0.068960	...	0.038813	0.003015	-0.074585	-0.018391	-0.026449	0.005867	0.051495	-0.009829	0.030009

2s completed at 12:20

PYTHON + Google Colab

1. Markdown



The screenshot shows a Google Colab notebook titled "day_1.ipynb". The notebook contains the following content:

```
[6] import pandas as pd
from sentence_transformers import SentenceTransformer
from transformers import pipeline
```

Feature Extraction

We will begin by extracting features (numerical representations) from the text data using the `sentence-transformers` package. We will use the following three sentences, stored as a list of strings, as input to the model:

```
[7] sentences = [
    "I feel great this morning",
    "I am feeling very good today",
    "I am feeling terrible"
]
```

We will use the `all-MiniLM-L6-v2` model to extract features from the sentences. The model will encode the sentences into a 384-dimensional vector representation. We will then print the features as a pandas dataframe for easy viewing.

```
26 # Load the pre-trained model
model = SentenceTransformer('all-MiniLM-L6-v2')

# Extract features
features = model.encode(sentences)

# Print the features as a pandas dataframe
pd.DataFrame(features, index=sentences)
```

Below the code, the output is displayed as a pandas DataFrame with 384 columns (indices 0 to 382) and 3 rows corresponding to the sentences:

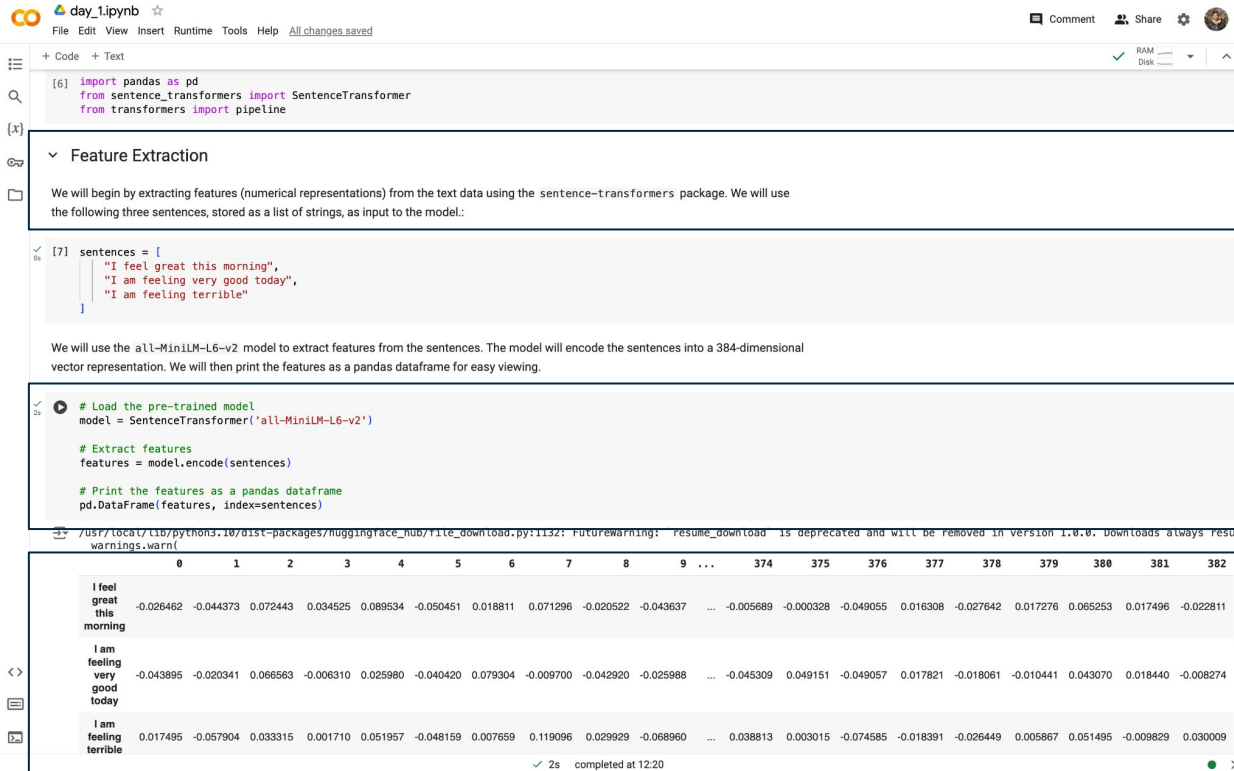
	0	1	2	3	4	5	6	7	8	9	...	374	375	376	377	378	379	380	381	382
I feel great this morning	-0.026462	-0.044373	0.072443	0.034525	0.089534	-0.050451	0.018811	0.071296	-0.020522	-0.043637	...	-0.005689	-0.000328	-0.049055	0.016308	-0.027642	0.017276	0.065253	0.017496	-0.022811
I am feeling very good today	-0.043895	-0.020341	0.066563	-0.006310	0.025980	-0.040420	0.079304	-0.009700	-0.042920	-0.025988	...	-0.045309	0.049151	-0.049057	0.017821	-0.018061	-0.010441	0.043070	0.018440	-0.008274
I am feeling terrible	0.017495	-0.057904	0.033315	0.001710	0.051957	-0.048159	0.007659	0.119096	0.029929	-0.068960	...	0.038813	0.003015	-0.074585	-0.018391	-0.026449	0.005867	0.051495	-0.009829	0.030009

The notebook interface shows the code was executed successfully, with a completion time of 2 seconds at 12:20.

2. Code

PYTHON + Google Colab

1. Markdown



The screenshot shows a Google Colab notebook titled "day_1.ipynb". The interface includes a menu bar (File, Edit, View, Insert, Runtime, Tools, Help) and a toolbar with icons for adding code or text, searching, and managing the notebook. The notebook content is divided into three sections:

- Code Section:** Contains two code cells. The first cell imports pandas, sentence-transformers, and transformers. The second cell defines a list of sentences:

```
sentences = ["I feel great this morning", "I am feeling very good today", "I am feeling terrible"]
```
- Markdown Section:** Titled "Feature Extraction", it contains two paragraphs. The first paragraph explains the goal of extracting features from text data using the sentence-transformers package. The second paragraph states that the all-MiniLM-L6-v2 model will be used to encode the sentences into a 384-dimensional vector representation.
- Code Section:** Contains a third code cell that loads the pre-trained model, extracts features, and prints them as a pandas dataframe:

```
# Load the pre-trained model
model = SentenceTransformer('all-MiniLM-L6-v2')

# Extract features
features = model.encode(sentences)

# Print the features as a pandas dataframe
pd.DataFrame(features, index=sentences)
```

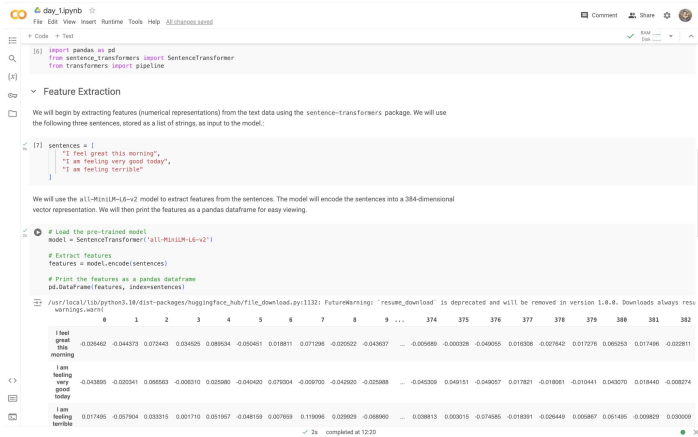
Below the code cells, a printout of the resulting pandas dataframe is shown. The dataframe has three rows corresponding to the sentences and 384 columns representing the vector features. The first row is for "I feel great this morning", the second for "I am feeling very good today", and the third for "I am feeling terrible". The printout is truncated in the middle of the columns.

	0	1	2	3	4	5	6	7	8	9	...	374	375	376	377	378	379	380	381	382
I feel great this morning	-0.026462	-0.044373	0.072443	0.034525	0.089534	-0.050451	0.018811	0.071296	-0.020522	-0.043637	...	-0.005689	-0.000328	-0.049055	0.016308	-0.027642	0.017276	0.065253	0.017496	-0.022811
I am feeling very good today	-0.043895	-0.020341	0.066563	-0.006310	0.025980	-0.040420	0.079304	-0.009700	-0.042920	-0.025988	...	-0.045309	0.049151	-0.049057	0.017821	-0.018061	-0.010441	0.043070	0.018440	-0.008274
I am feeling terrible	0.017495	-0.057904	0.033315	0.001710	0.051957	-0.048159	0.007659	0.119096	0.029929	-0.068960	...	0.038813	0.003015	-0.074585	-0.018391	-0.026449	0.005867	0.051495	-0.009829	0.030009

2. Code

3. Printouts

PYTHON + Google Colab



```
File Edit View Insert Runtime Tools Help 88.04000.0000
+ Code + Text
[5] import pandas as pd
from sentence_transformers import SentenceTransformer
from transformers import pipeline

Feature Extraction

We will begin by extracting features (numerical representations) from the text data using the sentence-transformers package. We will use the following three sentences, stored as a list of strings, as input to the model.

[7] sentences = [
    "I feel great this morning",
    "I am feeling very good today",
    "I am feeling terrible"
]

We will use the all-MiniLM-L6-v2 model to extract features from the sentences. The model will encode the sentences into a 384-dimensional vector representation. We will then print the features as a pandas dataframe for easy viewing.

# Load the pre-trained model
model = SentenceTransformer('all-MiniLM-L6-v2')

# Extract features
features = model.encode(sentences)

# Print the features as a pandas dataframe
pd.DataFrame(features, index=sentences)
```

`FutureWarning: 'resume_download' is deprecated and will be removed in version 1.0.0. Downloads always resume when possible. To avoid this warning, you can set the 'resume_download' parameter to False.`

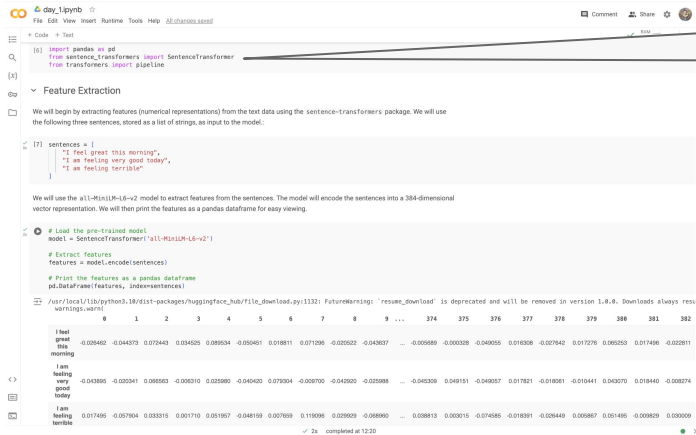
	0	1	2	3	4	5	6	7	8	...	374	375	376	377	378	379	380	381	382	
I feel great this morning	-0.026462	-0.044373	0.072943	0.034026	0.089534	-0.050461	0.018811	0.071296	-0.020522	-0.043037	...	-0.005089	-0.000328	-0.049005	0.016308	-0.027642	0.017276	0.060253	0.017406	-0.002811
I am feeling very good today	-0.043886	-0.020341	0.080903	-0.006210	0.025890	-0.040403	0.077604	-0.008700	-0.040262	-0.023588	...	-0.046330	0.068151	-0.048057	0.017821	-0.018061	-0.010441	0.040270	0.018440	-0.008274
I am feeling terrible	0.017495	-0.057604	0.033315	0.001710	0.051857	-0.048159	0.007769	0.119306	0.023929	-0.088900	...	0.008813	0.003015	-0.074585	-0.018391	-0.026440	0.005867	-0.051405	-0.006829	0.030008

2s completed at 12:20

PYTHON + Google Colab

package imports

```
import pandas as pd
from sentence_transformers import SentenceTransformer
from transformers import pipeline
```



```
6 day_1.pytnb
File Edit View Insert Runtime Tools Help
+ Code + Text
[5] import pandas as pd
from sentence_transformers import SentenceTransformer
from transformers import pipeline

Feature Extraction
We will begin by extracting features (numerical representations) from the text data using the sentence-transformers package. We will use the following three sentences, stored as a list of strings, as input to the model.

[7] sentences = [
    "I feel great this morning",
    "I am feeling very good today",
    "I am feeling terrible"
]

We will use the all-MiniLM-L6-v2 model to extract features from the sentences. The model will encode the sentences into a 384-dimensional vector representation. We will then print the features as a pandas dataframe for easy viewing.

# Load the pre-trained model
model = SentenceTransformer('all-MiniLM-L6-v2')

# Extract features
features = model.encode(sentences)

# Print the features as a pandas dataframe
pd.DataFrame(features, index=sentences)
```

FutureWarning: 'resume_download' is deprecated and will be removed in version 1.0.0. Downloads always resume when possible. To avoid this warning, you can pass the argument 'resume_download=False'.

	0	1	2	3	4	5	6	7	8	9	...	374	375	376	377	378	379	380	381	382
I feel great this morning	-0.026462	-0.044373	0.072943	0.034026	0.089304	-0.050461	0.018811	0.071296	-0.020522	-0.043037	...	-0.005889	-0.000328	-0.049005	0.016306	-0.027642	0.017276	0.060253	0.017406	-0.020811
I am feeling very good today	-0.043886	-0.020541	0.088903	-0.006210	0.025890	-0.040403	0.076004	-0.008700	-0.040262	-0.023588	...	-0.046309	0.068151	-0.048057	0.017821	-0.018061	-0.010441	0.040270	0.018440	-0.008274
I am feeling terrible	0.017495	-0.057604	0.033315	0.001710	0.051867	-0.048169	0.007669	0.119006	0.022929	-0.048900	...	0.008813	0.003015	-0.074585	-0.018391	-0.026440	0.005867	-0.051405	-0.008629	0.030008

2x completed at 12:20

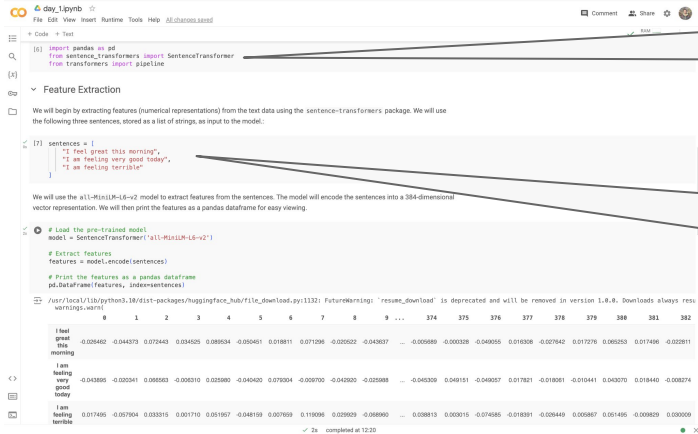
PYTHON + Google Colab

package imports

```
import pandas as pd
from sentence_transformers import SentenceTransformer
from transformers import pipeline
```

variable assignment, lists, strings

```
sentences = [
    "I feel great this morning",
    "I am feeling very good today",
    "I am feeling terrible"
]
```



```
day_1.py:nb
File Edit View Insert Runtime Tools Help
[15] import pandas as pd
from sentence_transformers import SentenceTransformer
from transformers import pipeline

Feature Extraction
We will begin by extracting features (numerical representations) from the text data using the sentence-transformers package. We will use the following three sentences, stored as a list of strings, as input to the model.

[17] sentences = [
    "I feel great this morning",
    "I am feeling very good today",
    "I am feeling terrible"
]

We will use the all-MiniLM-L6-v2 model to extract features from the sentences. The model will encode the sentences into a 384-dimensional vector representation. We will then print the features as a pandas dataframe for easy viewing.

[18] # Load the pre-trained model
model = SentenceTransformer('all-MiniLM-L6-v2')

# Extract features
features = model.encode(sentences)

# Print the features as a pandas dataframe
pd.DataFrame(features, index=sentences)
```

	0	1	2	3	4	5	6	7	8	...	374	375	376	377	378	379	380	381	382	
I feel great this morning	-0.036462	-0.044373	0.072943	0.034326	0.089534	-0.050461	0.018811	0.071296	-0.020522	-0.043037	...	-0.005889	-0.000328	-0.049005	0.016306	-0.027642	0.012726	0.060253	0.017406	-0.020811
I am feeling very good today	-0.043886	-0.020341	0.088953	-0.006210	0.025890	-0.040430	0.076204	-0.028700	-0.042620	-0.023588	...	-0.046330	0.068151	-0.048037	0.017821	-0.018061	-0.010441	0.040270	0.018440	-0.008274
I am feeling terrible	0.017495	-0.057904	0.033315	0.061710	0.051857	-0.048159	0.007659	0.119306	0.022929	-0.088900	...	0.028813	0.003015	-0.074585	-0.018391	-0.026440	0.005867	0.051405	-0.008629	0.030009

2x completed at 12:20

PYTHON + Google Colab

package imports

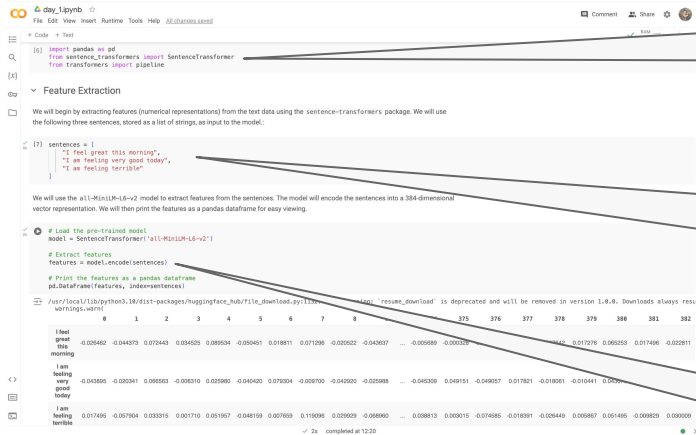
```
import pandas as pd
from sentence_transformers import SentenceTransformer
from transformers import pipeline
```

variable assignment, lists, strings

```
sentences = [
    "I feel great this morning",
    "I am feeling very good today",
    "I am feeling terrible"
]
```

dot notation, methods, attributes

```
# Extract features
features = model.encode(sentences)
```



The screenshot shows a Google Colab notebook with the following content:

```
File Edit View Insert Runtime Tools Help
+ Code + Text
[1] import pandas as pd
from sentence_transformers import SentenceTransformer
from transformers import pipeline

Feature Extraction
We will begin by extracting features (numerical representations) from the text data using the sentence-transformers package. We will use the following three sentences, stored as a list of strings, as input to the model.

[7] sentences = [
    "I feel great this morning",
    "I am feeling very good today",
    "I am feeling terrible"
]

We will use the all-MiniLM-L6-v2 model to extract features from the sentences. The model will encode the sentences into a 384-dimensional vector representation. We will then print the features as a pandas dataframe for easy viewing.

# Load the pre-trained model
model = SentenceTransformer('all-MiniLM-L6-v2')

# Extract features
features = model.encode(sentences)

# Print the features as a pandas dataframe
pd.DataFrame(features, index=sentences)
```

The output shows a pandas DataFrame with 3 rows and 384 columns. The first row corresponds to "I feel great this morning", the second to "I am feeling very good today", and the third to "I am feeling terrible". The values are numerical representations of the sentences.

PYTHON + Google Colab

package imports

```
import pandas as pd
from sentence_transformers import SentenceTransformer
from transformers import pipeline
```

variable assignment, lists, strings

```
sentences = [
    "I feel great this morning",
    "I am feeling very good today",
    "I am feeling terrible"
]
```

dot notation, methods, attributes

```
# Extract features
features = model.encode(sentences)
```

printing

```
# Print the features as a pandas dataframe
pd.DataFrame(features, index=sentences)
```

	0	1	2	3	4	5	6
I feel great this morning	-0.026462	-0.044373	0.072443	0.034525	0.089534	-0.050451	0.018811
I am feeling very good today	-0.043895	-0.020341	0.066563	-0.006310	0.025980	-0.040420	0.079304

OUR SOFTWARE STACK



+



+



Time to install our stack...



MAX PLANCK INSTITUTE
FOR HUMAN DEVELOPMENT



LLM4BeSci_StGallen2026 Public

Pin Watch 0 Fork 0 Star 1

main 1 Branch 0 Tags Go to file Add file Code

Zak-Hussain	remove cover image tag	a8eac46 · now	29 Commits
day_1	typos	2 days ago	
day_2	typo	yesterday	
day_3	fixing generation args	yesterday	
day_4	add clean_up_tokenization_spaces=False	23 minutes ago	
day_5	fix markdown	6 minutes ago	
.gitignore	Initial commit	last month	
LICENSE.txt	populate	last month	
README.md	remove cover image tag	now	
notes.txt	update	yesterday	

README License

LLM4BeSci at GSERM, St. Gallen 2026

The course introduces the use of open-source large language models (LLMs) from the Hugging Face ecosystem for research in the behavioral and social sciences.

By [Dirk Wulff](#) and [Zak Hussain](#)

Feedback

About

This course introduces the use of open-source large language models (LLMs) from the Hugging Face ecosystem for research in the behavioral and social sciences.

- Readme
- View license
- Activity
- 1 star
- 0 watching
- 0 forks

Releases

No releases published
[Create a new release](#)

Packages

No packages published
[Publish your first package](#)

Contributors 1

Zak-Hussain Zak Hussain

Languages

Jupyter Notebook 100.0%